Making the IISS a Progressive Web App (PWA)

Goals:

- Allow users to “install” the IISS app to their home screen
- In chrome, a pop-up appears to prompt this “install”

Why a PWA:

- Turning our web app into a PWA allows is to function like a native app
- The primary difference between a web app (what we had) and a **progressive** web app (which I changed it to) is that a PWA has a **Service Worker.**
- This service worker sits between the user’s browser and the network, acting as a programmable proxy server.
- This can be configured to give the web app the following functionalities (among others):
    - Installability
    - Offline capabilities
    - Push notification

*Note: For the purpose of this project, I only implemented the **Installability** feature, focusing on giving users an app-like experience (e.g. launching the app from their home screen with a standalone UI).*

What makes an app a PWA:

1. Served on HTTPS

    Required for the service worker. Our app was already running on HTTPS, so this step wasn’t a problem.
2. Has an active Service Worker
3. Has a properly configured Web App Manifest (`Manifest.js`)

If our app meets these 3 requirements it will have installability, meaning:

- Chrome and other browsers will display an **“Install” prompt**
- Users can **add the app to their home screen**
- When launched, the app will open in a **standalone window**, not in a browser tab
- The UI feels native

My Changes:

1. **Service Worker**

   To implement the service worker I edited `index.html` and created the file `serviceWorker.js`

   The following script was added to `index.html`. This registers the service worker. This lets the browser know that a service worker will be handling network requests and tells it where to look for said service worker:

```
  <script>

       if ('serviceWorker' in navigator) {
         window.addEventListener('load', function() {


navigator.serviceWorker.register('serviceWorker.js').then(function(registration
) {
           // registration was successful
           console.log('Service worker registration successful with scope: ',
registration.scope);
         }, function(err) {
           // registration failed
           console.log('Service worker registration failed: ', err);
         });
         });
       }

     </script>
```

Below is the Service Worker code in **`serviceWorker.js`**. This is the simplest amount of code necessary to have an active service worker. This code does not actually cache resources or interfere with network traffic, but it could easily be configured to do so in the future:

```
// INSTALL
self.addEventListener('install', function(event) {
 console.log('[Service Worker] Install');
 self.skipWaiting();
});


// ACTIVATE
self.addEventListener('activate', function(event) {
  console.log('[Service Worker] Activated');
 self.clients.claim();
});

// FETCH: Optional: Log fetches without interfering
self.addEventListener('fetch', function(event) {
 console.log('[Service Worker] Fetch:', event.request.url);
});
```

2. **Web App Manifest**

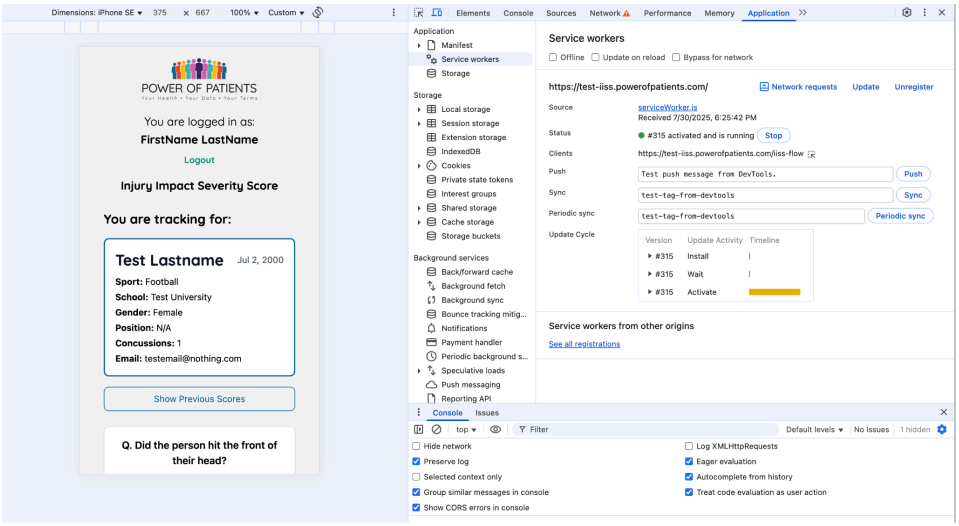
The primary aspect our **`Manifest.js`** file was lacking was properly sized icons. These are the icons displayed on the home screen once the user “installs”. A PWA requires one 512x512 px icon and one 192x192px icon. I created icons of the specific sizes, added them to the project and added lines 4-13 of **`Manifest.js:`**

```
{
 "short_name": "IISS",
 "name": "Injury Impact Severity Score",
 "icons": [
    {
     "src": "icons/icon-192x192.png",
     "type": "image/png",
     "sizes": "192x192"
   },
   {
```

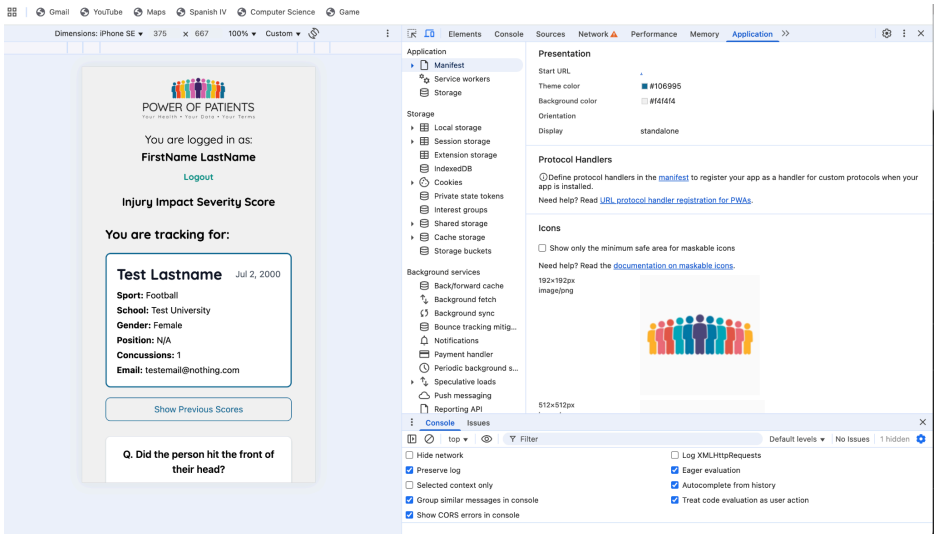
```
    "src": "icons/icon-512x512.png",
    "type": "image/png",
    "sizes": "512x512"
  },
  {
    "src": "favicon.ico",
    "sizes": "64x64",
    "type": "image/x-icon"
  }
],
"start_url": ".",
"display": "standalone",
"theme_color": "#106995",
"background_color": "#f4f4f4"
}
```

Testing:

Service Worker is running:



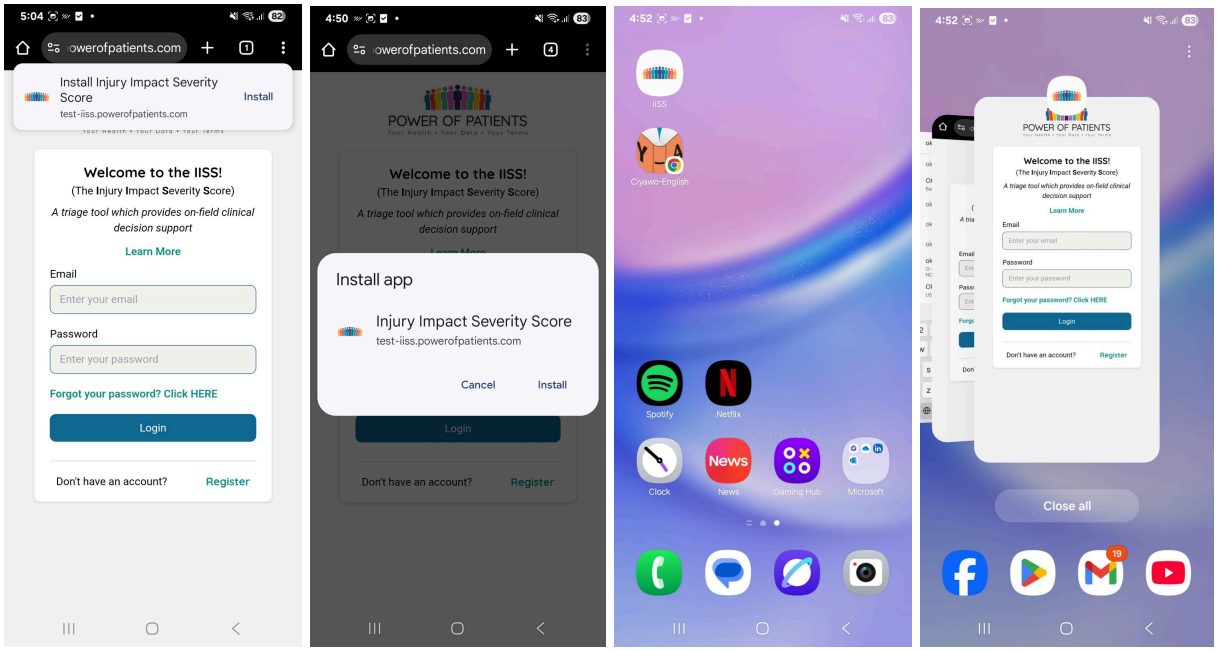
Properly Finding Web App Manifest Data:



Demo:

On Android:

Below you can see how the user can Install the App to their home screen (from Chrome). The goal of installability and a native-app-like experience was achieved:



On iPhone:

The same principle applies. The only difference here is that the browser I am using doesn't have the pop-up feature the Chrome does for PWAs, so you have to use the "Add to Home Screen" button: